

Project 3: Efficient Fine-Tuning with LoRA

Max Goldmuntz
Computer Science Department, Brandeis University

December 19, 2025

1 Preprocessing Steps and Rationale

Before fine-tuning, the data was processed to ensure compatibility with the models and optimal training stability.

1.1 Task 1: QA Preprocessing (RoBERTa)

- **Tokenization Alignment:** The SQuAD dataset provides answers as character indices (e.g., "starts at char 45"), but the model sees tokens. We used the 'offset_mapping' feature of the tokenizer to map character positions to token boundaries.
- **Truncation and Padding:** Sequences were truncated to a maximum length of 384 tokens with a stride of 128. This stride ensures that if an answer spans across a split point (due to a long document), it is not lost but captured in the overlapping window of the next sequence.

1.2 Task 2: Code Preprocessing (Mellum-4B)

- **Instruction Formatting:** The raw dataset contained separate columns for "instruction" and "output". These were merged into a single prompt format: "Instruction: [text] Response: [code]" to teach the model the causal relationship between the query and the code.
- **Dynamic Padding:** Instead of padding all sequences to the maximum length (which wastes GPU memory on zeros), we used a data collator to dynamically pad each batch to the length of the longest sequence in that specific batch. This significantly sped up training by reducing computational overhead.

2 Training and Validation Loss Curves

We tracked the loss dynamics to monitor convergence and prevent overfitting.

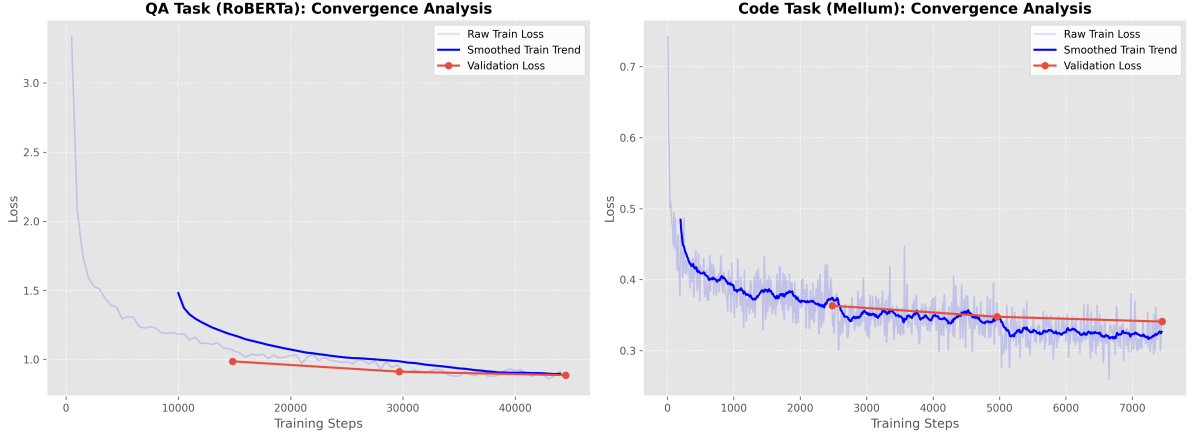


Figure 1: Left: RoBERTa (QA) loss curve shows rapid convergence. Right: Mellum-4B (Code) shows a steady, non-volatile descent, indicating a stable learning rate ($2e - 4$) and effective gradient accumulation.

3 Experimental Results and Analysis

3.1 Experiment 1: Data Efficiency (QA)

We hypothesized that pre-trained models require minimal data to adapt to extractive QA.

- **30% Data:** 68.21% Exact Match
- **50% Data:** 71.33% Exact Match
- **100% Data:** 73.66% Exact Match

Analysis: The results confirm high data efficiency. The model recovered 92% of its full capability using only 30% of the data. This suggests that the "knowledge" of how to answer questions is largely present in the pre-trained weights, and LoRA simply learns the "interface" or format of the SQuAD task.

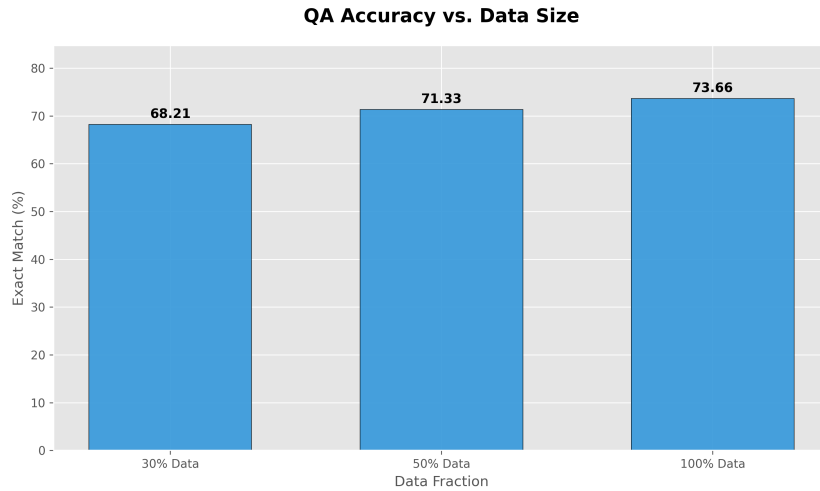


Figure 2: QA Accuracy vs. Dataset Size.

3.2 Experiment 2: Rank Sensitivity (QA)

We also investigated whether increasing the rank (r) improves performance for the QA task. As shown below, performance plateaued quickly, with Rank 16 (74.31%) and Rank 32 (74.29%) performing nearly identically. This reinforces the finding that the adaptation required for SQuAD is low-rank in nature.

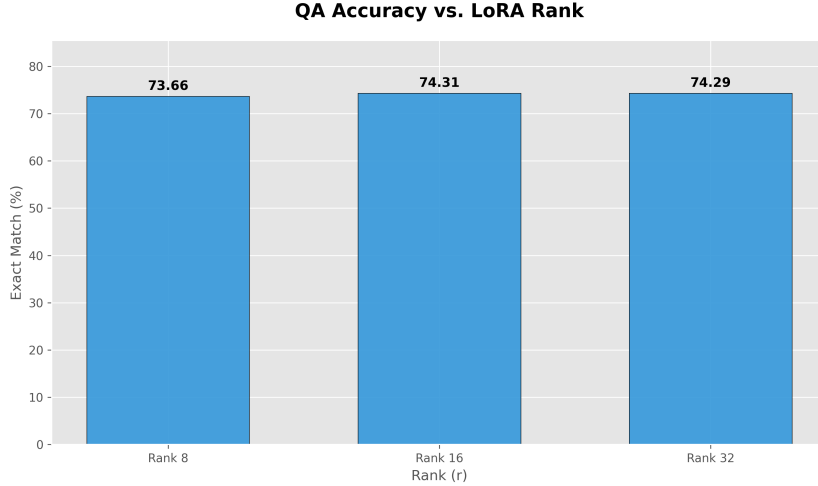


Figure 3: QA Accuracy vs. LoRA Rank. Performance is stable across ranks.

3.3 Experiment 3: Rank Saturation (Code Generation)

For the code generation task, we compared the pre-trained Base Model against LoRA adapters of varying ranks.

- **Base Model:** BLEU 0.81, Executability 52%
- **LoRA (All Ranks):** BLEU 0.90, Executability 50%

Analysis:

1. **Style Transfer Success:** The jump in BLEU ($0.81 \rightarrow 0.90$) indicates the model successfully adapted to the specific syntax and formatting of the target dataset.
2. **Rank Saturation:** A critical finding was that Ranks 8, 16, and 32 produced *identical* code outputs under greedy decoding (despite slightly lower training loss for Rank 32, see Figure 4). This implies the adaptation task has a low intrinsic dimension; the optimization path for learning Python syntax is simple enough that even a low-rank matrix ($r = 8$) captures it fully.

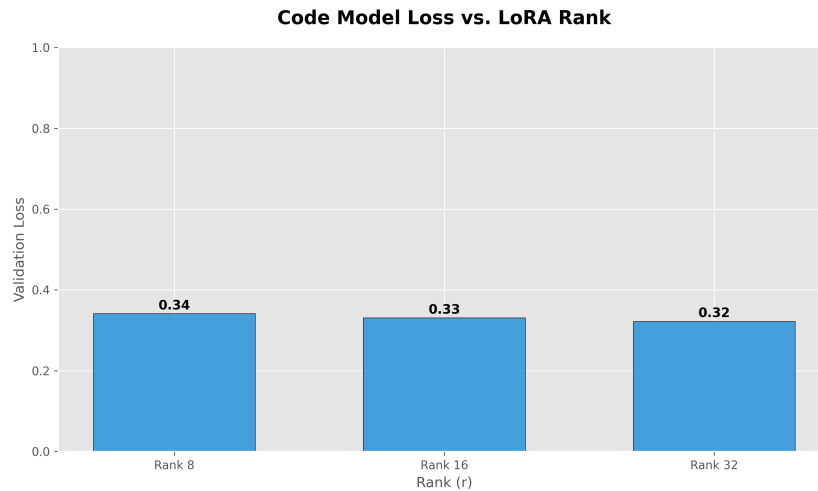


Figure 4: Code Model Loss vs. LoRA Rank.

Summary of Experimental Results

Experiment	Condition	Result
QA Size Exp	30% Data	68.21%
QA Size Exp	50% Data	71.33%
QA Size Exp	100% Data	73.66%
QA Rank Exp	Rank 16	74.31%
QA Rank Exp	Rank 32	74.29%
Code Rank Exp	Rank 8	0.3410
Code Rank Exp	Rank 16	0.3306
Code Rank Exp	Rank 32	0.3218

Figure 5: Summary of experimental results across all tasks and hyperparameters.

4 Semester Reflection: What I Learned

Over the course of these three projects, I have developed a holistic understanding of the NLP pipeline.

1. **From Mechanics to Abstractions:** In Project 1 (N-Grams), I learned the raw mechanics of probability in language. Project 2 (RNNs/LSTMs) taught me about sequence modeling and the vanishing gradient problem. Finally, Project 3 (Transformers & LoRA) showed me how modern architectures solve these issues using attention mechanisms, allowing for massive parallelization that was impossible with RNNs.

2. **The Cost of Scale:** A recurring theme was the trade-off between performance and compute. In Project 2, training simple RNNs was slow due to their sequential nature. In Project 3, I learned that we don't need to retrain massive models from scratch; techniques like LoRA allow us to adapt billion-parameter models on a single GPU by training less than 1% of the parameters.
3. **Data vs. Architecture:** Across all projects, I found that data quality often mattered more than architectural tweaks. In this final project, simply formatting the data correctly (instruction tuning) provided the base performance, while hyperparameter tuning (Rank) had a negligible effect compared to the presence of the data itself.